

CSI247:
Data Structures

Lecture 03 – Java methods

Department of Computer Science
B. Gopolang (247-275)



Previous Lesson …

- Review of some Java Programming basics
- Java defined Classes and Objects
- Java API documentation
- Keyboard input
- File IO

UNIVERSITY of BOTSWANA

Today …



Learning Objectives

By the end of this lecture, a learner must be able to:

- Implement a method
- Invoke a method
- Understand parameter passing
- Determine scope of a variable in relation to a method

1. What is a method – a review

- A block of related statements that:
  - Serves one task
    - E.g.:
      - A method that finds the average of 2 numbers
  - Statements placed between '{' and '}'

- Each has:
  - A **name**:
    - Follow the same rules as variable names
  - () at the end
    - A place to receive input

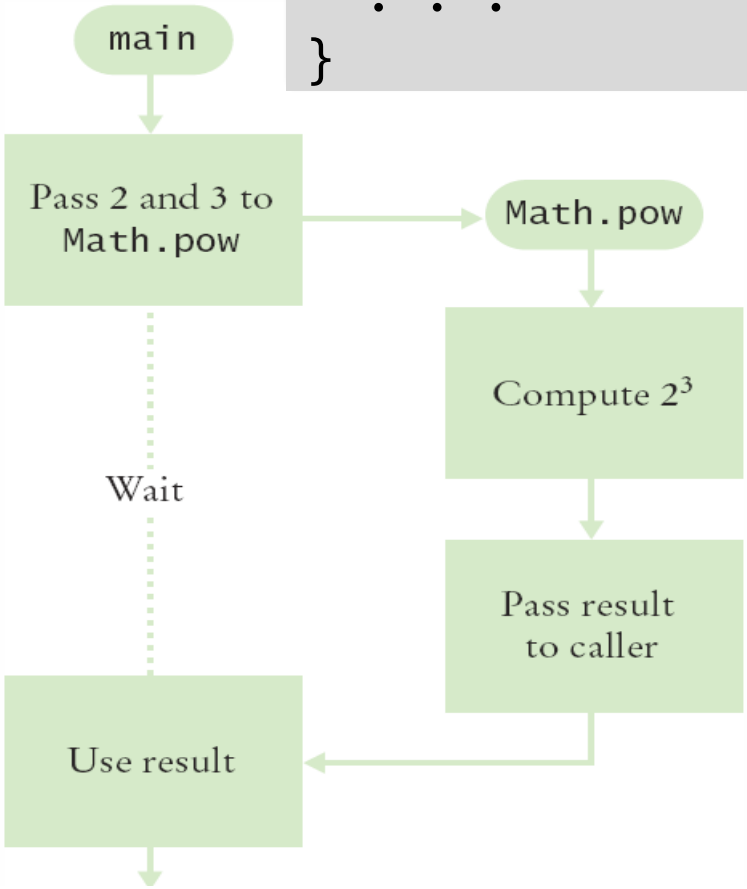
2. Examples of methods

- Some of the methods you have already used:
  - `main()`
  - `nextInt(), next() – from Scanner class`
  - `length(), charAt() – from String class`
  - `Math.max()`
  - `Integer.parseInt()`

- Note:
  - Methods differ
    - Some **give** something to the environment only
    - Some **take** something from the environment only
    - Others do both (**give & take)**

3. Flowchart of calling a Method

```
public static void main(String[] args){
  double z = Math.pow(2, 3);
  . . .
}
```

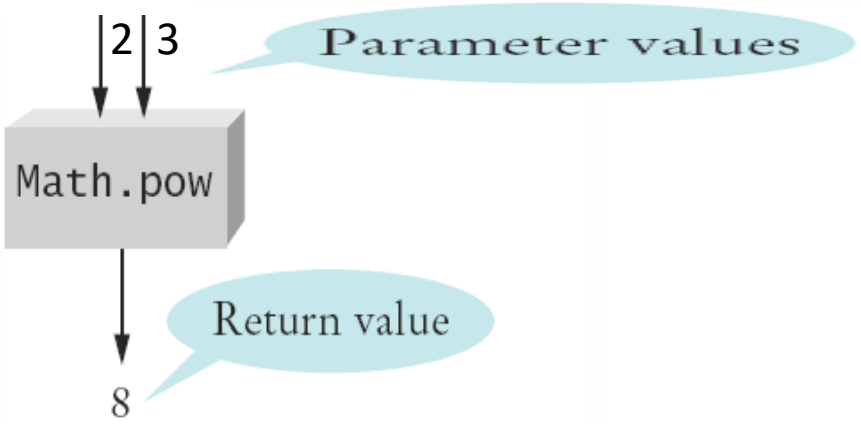


One method 'calls' another

- **main()** calls **Math.pow()**
  - Passes two values (2 & 3) to Math.pow
  - Math.pow starts
    - Uses information passed (2 & 3)
    - Does its job
    - Returns the answer to main()
- main uses results

4. Parameters and Return Values

```
public static void main(String[] args){
   double z = Math.pow(2, 3);
   . . .
}
```



- **a) main** "passes" two parameters to Math.pow
- **b) Math.pow** calculates and returns a value of 8 to main
- **c) main** stores the return value in variable z

5. Implementing Methods



- Assume a method that calculate volume of a cube
  - What does it need to do its job?
  - What does it answer with?

- When writing this method:
  - Pick a **name** for the method (e.g. **cubeVolume**).
  - Give a **type and name for each parameter** if any (**double length**)
  - Specify the **type of the return value** if any (**double**)
  - Add modifiers such as **public, static,** etc

```
public static double cubeVolume(double length)
```

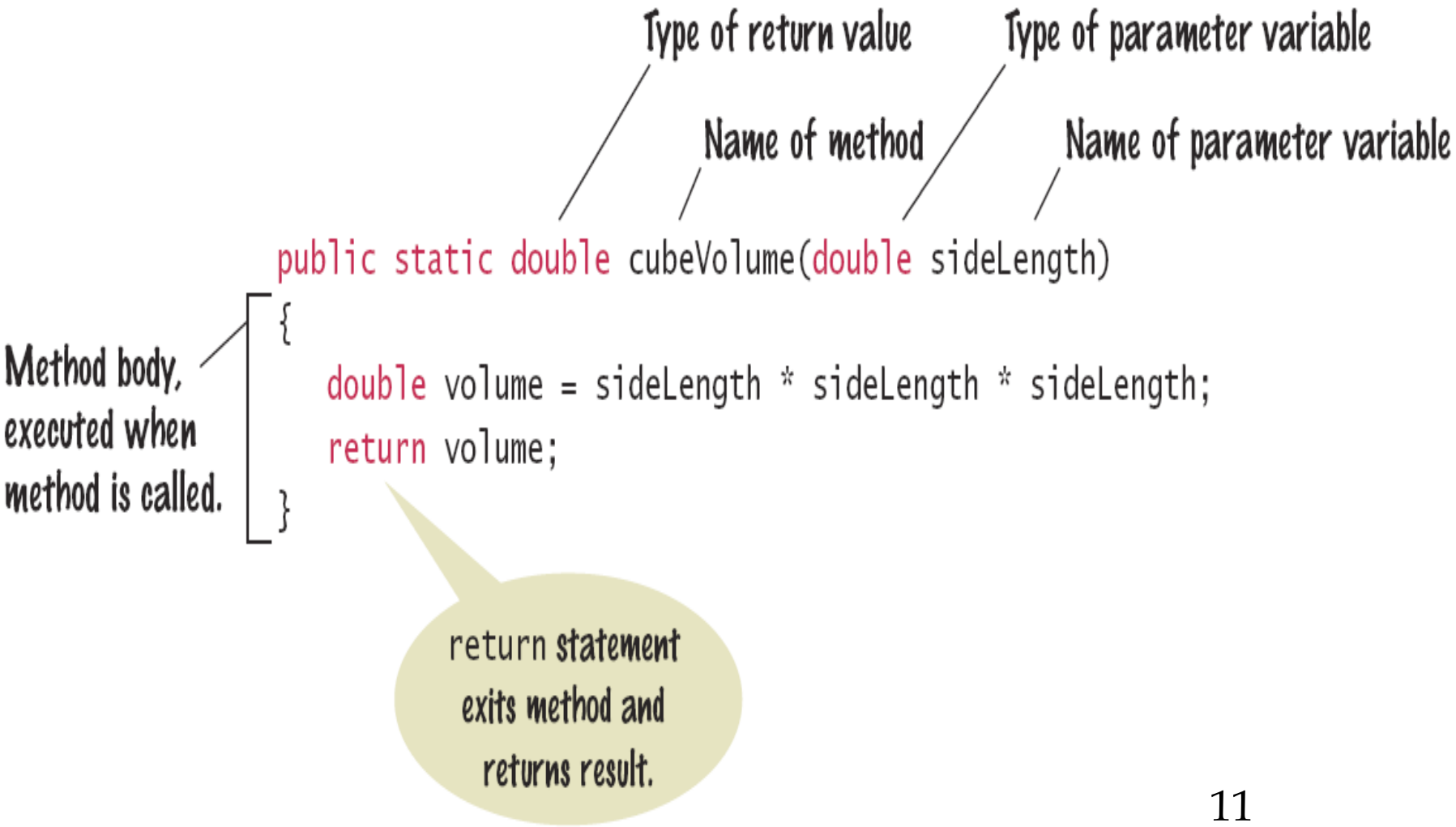
6. Inside the Method

- Referred to as **body** of the method
  - The **body** is
    - Surrounded by curly braces { }
    - contains variable declarations and statements that are executed when the method is called/invoked
    - will also **return** the calculated answer if needed

- E.g.

```
public static double cubeVolume(double length){
    double volume = length * length * length;
    return volume;
}
```

7. Method Declaration



8. Invoking/calling the method

- E.g.

```
public static void main(String[] args) {
  double result1 = cubeVolume(2);
  double result2 = cubeVolume(10);
  System.out.printf("A cube of length 2 has volume %f",
   result1);
  System.out.printf("A cube of length 10 has volume %f",
   result2);
}
```

- Values returned from a method are used in the calling environment
  - Here: stored in local variables result1 and result2
  - Then printed out



Cubes.java: complete program

```
1  /**
2     This program computes the volumes of two cubes.
3  */
4  public class Cubes
5  {
6     public static void main(String[] args)
7     {
8        double result1 = cubeVolume(2);
9        double result2 = cubeVolume(10);
10       System.out.println("A cube with side length 2 has volume " + result1);
11       System.out.println("A cube with side length 10 has volume " + result2);
12    }
13
14    /**
15       Computes the volume of a cube.
16       @param sideLength the side length of the cube
17       @return the volume
18    */
19    public static double cubeVolume(double sideLength)
20    {
21       double volume = sideLength * sideLength * sideLength;
22       return volume;
23    }
24 }
```

Program Run

```
A cube with side length 2 has volume 8
A cube with side length 10 has volume 1000
```

9. Method comments

- Start with /** (Javadoc multiline comment)
  - Note the purpose of the method
  - Describe each parameter & the return value
- End with */

```
/**
  Computes the volume of a cube.
  @param sideLength the side length of the cube
  @return the volume
*/
public static double cubeVolume(double sideLength){
    . . .
}
```

10. Repeating code as a method

- Find Repetitive Code – candidates for a method!!
  - May have different values but same logic

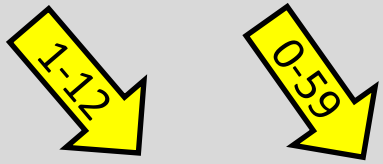
1 - 12

0 - 59

```
int hours;
System.out.print("Enter a value between 1 and 12: ");
hours = in.nextInt();
while (hours < 1 || hours > 12) {
  System.out.print("Enter a value between 1 and 12: ");
  hours = in.nextInt();
}
//
int minutes;
System.out.print("Enter a value between 0 and 59: ");
minutes = in.nextInt();
while (minutes < 0 || minutes > 59) {
  System.out.print("Enter a value between 0 and 59: ");
  minutes = in.nextInt();
}
```

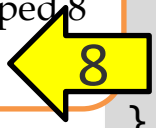
11. Write a 'Parameterized' Method

```
/**
   Prompts a user to enter a value in a given range until the
    user provides a valid input.
   @param low the low end of the range
   @param high the high end of the range
   @return the value provided by the user
*/
```



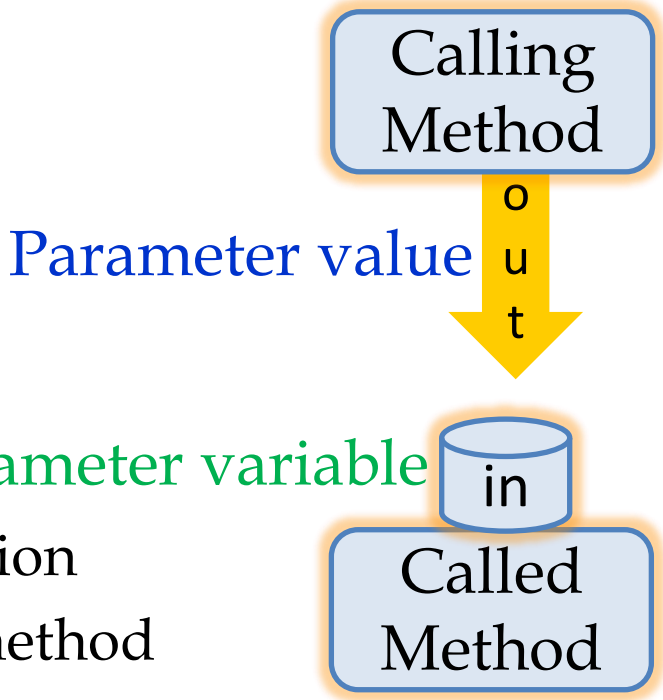

```
public static int readValueBetween(int low, int high) {
   Scanner in = new Scanner(System.in);
   int input = 0;
   while (input < low || input > high) {
      System.out.printf("Enter value between %d and %d: " ,low , high);
      input = in.nextInt();
   }
   return input;
}
```

Assumption: user typed 8



12. Parameter Passing

- Parameter variables hold the parameter values supplied in the method call:
  - Both must be of the same data type
- The parameter value may be:
  - The contents of a variable
  - A 'literal' value (e.g. 2)
  - aka. 'actual parameter' or argument
- The parameter variable is:
  - Named in the called method declaration
  - Used as a variable inside the called method
  - aka. 'formal parameter'



Parameter Passing Steps

```
public static void main(String[] args)
{
   double result1 = cubeVolume(2);
   result1 = 8
   . . .
}
```

```
public static double cubeVolume(double sideLength)
{
   double volume = sideLength * sideLength * sideLength;
   return volume;
}
```

sideLength = 2

volume = 8

Common Error 1

- Trying to Modify Parameters
  - Copy of parameter value is passed
  - Called method (addTax) can modify its local copy (price)
    - Not original copy (in calling method)

```
public static void main(String[] args{
   double t, total = 10;
   t = addTax(total, 7.5);
   // print total here!
}
```

total 10.0

Copy of total

```
public static double addTax( double price, double rate){
   double tax = price * rate / 100;
   price = price + tax; // Has no effect outside the method
   return tax;
}
```

price 10.75

13. Methods with Return Values

- Methods can (optionally) return one value
  - The return type is specified in the method declaration
  - The return statement does two things:
    1) The method terminates immediately
    2) The return value is returned to the calling method

Return type

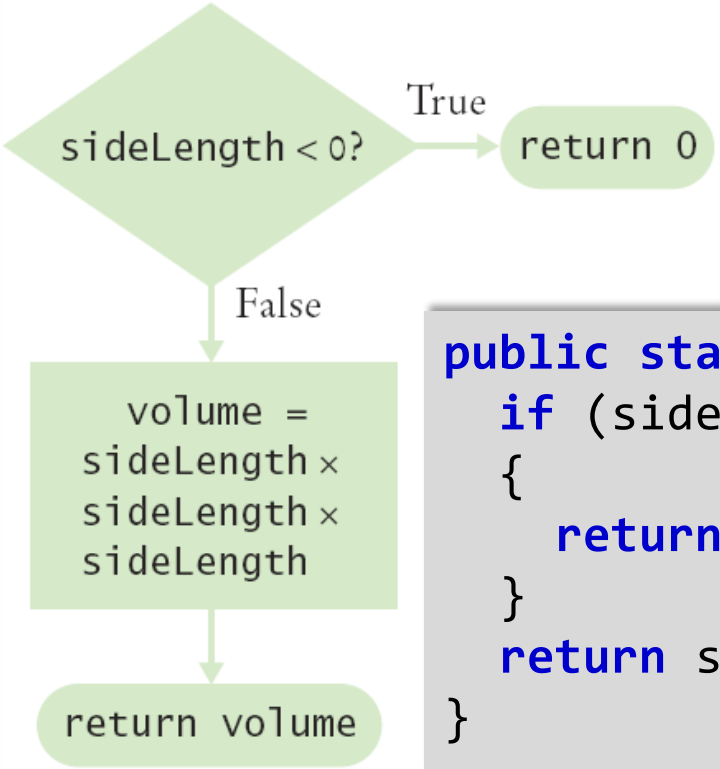
```
public static double addTax(double price, double rate){
  price += price * rate / 100;
  return price;
}
```

Return value (May be a value, a variable or a calculation)

- **Return value data type MUST match return type**

14. Multiple return Statements

- A method can use multiple return statements
  - But Every branch must have a return statement




```
public static double cubeVolume(double sideLength){
  if (sideLength < 0)
  {
    return 0;
  }
  return sideLength * sideLength * sideLength;
}
```

Common Error 2

- **Missing return Statement**
  - E.g.

```
public static int sign(double x){
   if (x < 0) {
      return -1;
   }
   if (x > 0) {
      return 1;
   }
   // Error: missing return value for x=0
}
```

  - In this case, x may be equal to 0
  - Compiler complains if any branch has no return statement
    - Make sure all conditions are handled

15. Steps to implement a Method

1) Describe what the method should do
2) Determine the method's inputs, if any
3) Determine data types: inputs and the return value, if any
4) Write pseudocode for the desired result
5) Implement the method body

```
public static double pyramidVolume( double h, double b){
  double baseArea = b * b;
  return h * baseArea / 3;
}
```

6) Test the method
   - Design test cases first

```
Volume: 300
Expected: 300
Volume: 0
Expected: 0
```

16. Methods without Return Values

- It is NOT compulsory for methods to return a value
  - The return type of **void** means nothing is returned
  - **No return statement is required**
  - The method may generate output though!

```
...
boxString("Hello");
...
```

```
-------
!Hello!
-------
```

```
public static void boxString(String str){
  int n = str.length();
  for (int i = 0; i < n + 2; i++)
    { System.out.print("-"); }
  System.out.println();
  System.out.println("!" + str + "!");
  for (int i = 0; i < n + 2; i++)
    { System.out.print("-"); }
  System.out.println();
}
```

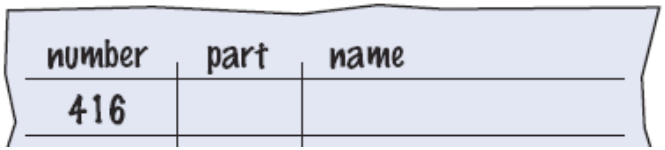
17. Using return Without a Value

- You can use the return statement without a value
  - In methods with **void** return type
  - The method will terminate immediately!

```
public static void boxString(String str) {
  int n = str.length();
  if (n == 0)
  {
    return; // Return immediately
  }
  for (int i = 0; i < n + 2; i++) { System.out.print("-"); }
  System.out.println();
  System.out.println("!" + str + "!");
  for (int i = 0; i < n + 2; i++) { System.out.print("-"); }
  System.out.println();
}
```

Programming Tips

- Keep methods short
  - Generally, one screen display
  - Break into 'sub' methods if code is too long
- Trace your methods



- Use **Stubs** when you write larger programs
  - Unfinished methods that return a 'dummy' value

```
public static String digitName(int digit){
  return "mumble";
}
```

18. Summary: Methods

- Method➔ a named sequence of instructions.
- Parameter values: supplied when a method is called.
- The return value ➔ the result that the method computes.
- You need these when declaring a method
  - Method name, a name and type for each parameter, and data type for the result, if any.
- Method comments explain the purpose of the method, the meaning of the parameters and return value, as well as any special requirements.

- Parameter variables hold the parameter values supplied in the method call.
- The **return** statement terminates a method call and sends back the method result.
  - Turn computations that can be reused into methods.
  - Use a return type of void to indicate that a method does not return a value.
  - Use the process of stepwise refinement to decompose complex tasks into simpler ones.

19. Variable Scope

Scope of a variable:
This is part of a program in which a variable is visible.

- Variables can be declared:
  a) Inside a method
    - Known as 'local variables'
    - Only available inside this method
    - Parameter variables are like local variables
  b) Inside a block of code { }
    - Sometimes called 'block scope'
    - Declared inside block { ends at end of block }
  c) Outside of a method
    - Sometimes called 'global scope'
    - Can be used (and changed) by code in any method
- How do you choose?

Examples of Scope

```
public static void main(String[] args){
   int sum = 0;
   for (int i = 1; i <= 10; i++)
   {
      int square = i * i;
      sum = sum + square;
   }
   System.out.println(sum);
}
```

square

i

sum

- **sum** : local variable in main
- **i** : only visible inside the for-loop
- **Square:** only visible inside the for-loop block

Local Variables of Methods

```
public static void main(String[] args){
  double sideLength = 10;
  int result = cubeVolume();
  System.out.println(result);
}
public static double cubeVolume(){
  return sideLength * sideLength * sideLength; // ERROR !
}
```

- Variables declared inside one method are not visible to other methods
  - sideLength is local to main
    - It is **unknown in cubeVolume** in this case
  - This will cause a compiler error

Re-using names for local variables

```
public static int square(int n) {
  int result = n * n;
  return result;
}

public static void main(String[] args){
  int result = square(3) + square(4);
  System.out.println(result);
}
```

result

result

- Variables declared inside one method are not visible to other methods
  - **result** is local to square and **result** is local to main
  - They are two different variables and **do not overlap**

Re-using names for block variables

```
public static void main(String[] args) {
  int sum = 0;
  for (int i = 1; i <= 10; i++) {
    sum = sum + i;
  }
  for (int i = 1; i <= 10; i++) {
    sum = sum + i * i;
  }
  System.out.println(sum);
}
```

- Variables declared inside one block are not visible to other blocks
  - i is inside the first for block and i is inside the second
  - They are two different variables, and do not overlap

Overlapping Scope

```
public static int sumOfSquares(int n) {
  int sum = 0;
  for (int i = 1; i <= n; i++) {
    int n = i * i; // ERROR
    sum = sum + n;
  }
  return sum;
}
```

block scope n

Local n

- Variables (including parameter variables) must have unique names within their scope
  - **n** has local scope and **n** is in a block inside that scope
  - The compiler will complain when the block scope **n** is declared

Global and Local Overlapping

```
public class Scoper {
  public static int same;   // 'global'
  public static void main(String[] args) {
    int same = 0;      // local
    for (int i = 1; i <= 10; i++){
      int square = i * i;
      same = same + square;
    }
    System.out.println(same);
  }
}
```

same

same

Variables in different scopes with the same name will compile, but it is not a good idea

- Global and Local (method) variables can overlap
  - The local same will be used when it is in scope
  - No access to global same when local same is in scope



20. Summary: Scope

- Local variables declared inside one method are not visible to code inside other methods
- The scope of a variable is the part of the program in which it is visible.
  - Two local or parameter variables can have the same name, provided that their scopes do not overlap.
  - You can use the same variable name within different methods since their scope does not overlap.



Next Lesson …

- Java Classes and Objects

Q & A

